

Beyond Model Scale Limits: End-Edge-Cloud Federated Learning with Self-Rectified Knowledge Agglomeration

Zhiyuan Wu, *Member, IEEE*, Sheng Sun, Yuwei Wang, *Member, IEEE*, Min Liu, *Senior Member, IEEE*,
Ke Xu, *Fellow, IEEE*, Quyang Pan, Bo Gao, *Member, IEEE*, and Tian Wen

Abstract—The rise of End-Edge-Cloud Collaboration (EECC) offers a promising paradigm for Artificial Intelligence (AI) model training across end devices, edge servers, and cloud data centers, providing enhanced reliability and reduced latency. Hierarchical Federated Learning (HFL) can benefit from this paradigm by enabling multi-tier model aggregation across distributed computing nodes. However, the potential of HFL is significantly constrained by the inherent heterogeneity and dynamic characteristics of EECC environments. Specifically, the uniform model structure bounded by the least powerful end device across all computing nodes imposes a performance bottleneck. Meanwhile, coupled heterogeneity in data distributions and resource capabilities across tiers disrupts hierarchical knowledge transfer, leading to biased updates and degraded performance. Furthermore, the mobility and fluctuating connectivity of computing nodes in EECC environments introduce complexities in dynamic node migration, further compromising the robustness of the training process. To address multiple challenges within a unified framework, we propose End-Edge-Cloud Federated Learning with Self-Rectified Knowledge Agglomeration (FedEEC), which is a novel EECC-empowered FL framework that allows the trained models from end, edge, to cloud to grow larger in size and stronger in generalization ability. FedEEC introduces two key innovations: (1) Bridge Sample Based Online Distillation Protocol (BSBDP), which enables knowledge transfer between neighboring nodes through generated bridge samples, and (2) Self-Knowledge Rectification (SKR), which refines the transferred knowledge to prevent suboptimal cloud model optimization. The proposed framework effectively handles both cross-tier resource heterogeneity and effective knowledge transfer between neighbor-

ing nodes, while satisfying the migration-resilient requirements of EECC. Extensive experiments on three datasets demonstrate that FedEEC achieves significantly higher cloud model accuracy compared with state-of-the-art methods.

Index Terms—Federated learning, edge computing, cloud computing, interaction protocol

I. INTRODUCTION

WITH the growing boom of Internet of Things (IoT), 6G, and Artificial Intelligence (AI), the proliferation of intelligent devices is accelerating at an exponential rate. According to the predictions from Statista, the number of IoT connections worldwide will reach 32.1 billion by 2030 [1]. This surge is accompanied by an unprecedented volume of data generated at the network end, offering immense potential for training advanced AI models and enhancing user experiences across a wide range of applications [2]. However, concerns over data privacy and ownership make it impractical to conduct centralized end-side data processing on the cloud, resulting in isolated data islands [3] that impede the development of comprehensive AI models and large-scale data-driven insights. To address this challenge, Federated Learning (FL) has emerged as a promising solution within the realm of Privacy Computing (PC), allowing multiple devices to jointly train machine learning models without sharing on-device data with the cloud [4]. In conventional FL [5]–[7], the server and end devices (clients) collaborate through iterative rounds of model parameters aggregation and distribution, effectively bridging isolated data islands while maintaining privacy guarantees.

Alongside the evolution of FL, the computing paradigm is shifting from centralized cloud-based processing to End-Edge-Cloud Collaboration (EECC) [8], where computational tasks are distributed across cloud servers, edge servers, and end devices. This paradigm offers reduced network latency, lower bandwidth consumption, and enhanced system reliability [9]. FL can benefit from the EECC paradigm to fully exploit pervasive devices and rapidly access large amounts of end-side data [10]. However, prevailing FL methods [5]–[7] adopt a simple client-server architecture with two tiers, where the server is either on the cloud or the edge. Such methods lack the architectural flexibility to fully exploit the potential of EECC environments, which require multi-tier collaboration across end devices, edge nodes, and cloud servers.

To overcome this limitation, Hierarchical Federated Learning (HFL) [11] is proposed to extend conventional FL by

Zhiyuan Wu, Quyang Pan and Tian Wen are with the State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, and also with the University of Chinese Academy of Sciences, Beijing, China. E-mails: wuzhiyuan22s@ict.ac.cn, panquyang23s@ict.ac.cn, wentian24s@ict.ac.cn.

Sheng Sun and Yuwei Wang are with the State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China. E-mails: {sunsheng, ywwang}@ict.ac.cn.

Min Liu is with the State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, and also with the Zhongguancun Laboratory, Beijing, China. E-mail: liumin@ict.ac.cn

Ke Xu is with the Department of Computer Science and Technology, Tsinghua University, Beijing, China, and also with the Zhongguancun Laboratory, Beijing, China. E-mail: xuke@tsinghua.edu.cn.

Bo Gao is with the School of Computer Science and Technology, and the Engineering Research Center of Network Management Technology for High-Speed Railway of Ministry of Education, Beijing Jiaotong University, Beijing, China. E-mail: bogao@bjtu.edu.cn.

Corresponding author: Yuwei Wang.

A preliminary version of this paper titled "Agglomerative Federated Learning: Empowering Larger Model Training via End-Edge-Cloud Collaboration" was accepted by IEEE International Conference on Computer Communications (INFOCOM), 2024. This work was supported by the National Key Research and Development Program of China (No. 2023YFB2703701), the National Natural Science Foundation of China (No. 62472410).

enabling multi-tier model aggregation across different tiers of computing nodes, unlocking the potential for efficient collaboration in an EECC system. Despite this promising potential, fully realizing the benefits of HFL in EECC environments is far from straightforward. The heterogeneous and dynamic nature of an End-Edge-Cloud Network (EEC-NET) introduces several interrelated challenges that hinder collaborative training across tiers.

The first challenge arises from computing power heterogeneity across end, edge, and cloud nodes. Specifically, cloud servers are equipped with powerful computing capabilities and storage resources, which outperform edge servers, and end devices possess relatively constrained resources [12]. Current HFL methods [11], [13]–[15] require imposing the same model structure on all computing nodes for model aggregation, inevitably limiting the scale of trained models by the least powerful end devices due to the bottleneck effect and causing resource under-utilization over edge and cloud computing nodes. The second challenge arises from the necessity for effective model update transfer from lower- to upper-tier nodes. In the end-edge-cloud hierarchy, the model updates learned by lower-tier devices must be effectively transferred upwards to ensure the performance of the cloud model. Due to the variations in data distributions and resource constraints across tiers, the model updates learned at lower tiers might be biased or incomplete, potentially introducing misleading patterns that propagate upward through the network hierarchy, ultimately resulting in suboptimal optimization of the cloud model. The third challenge arises from the introduction of the end-edge-cloud hierarchy, where computing nodes may need to switch between different upper-tier servers (called dynamic node migration) due to factors such as mobility or unstable connections [16], [17]. Some HFL methods [13], [18], with their static coordination algorithms, are unable to handle such dynamic node associations, resulting in reduced robustness in real-world deployments.

To effectively address the aforementioned challenges posed by the heterogeneous and dynamic nature of EECC, several key issues should be considered to guide the design of HFL. First, model sizes must be adapted to match the computational capabilities of nodes at different tiers, allowing resource-rich edge and cloud nodes to host larger models without being constrained by the capabilities of end devices. Second, the quality of exchanged information between neighboring nodes must be optimized, ensuring that every model update transferred from lower-tier nodes contributes to the enhancement of models at higher tiers rather than degrades their performance. Finally, it is crucial to enable seamless interaction between nodes with arbitrary model structures, ensuring compatibility with the dynamic migration of computing nodes commonly observed in EECC environments.

To this end, we propose a novel HFL framework suitable for EECC paradigm, named End-Edge-Cloud Federated Learning with Self-Rectified Knowledge Agglomeration (FedEEC). FedEEC is designed to enable recursive collaboration across computing nodes from end devices to edge and cloud servers, with model size and generalization ability increasing progressively from lower-tier to higher-tier nodes.

Specifically, FedEEC introduces two core innovations: Bridge Sample-Based Online Distillation Protocol (BSBODP) and Self-Knowledge Rectification (SKR). BSBODP tackles the challenge of heterogeneous model collaboration by facilitating model-agnostic [19] knowledge transfer between neighboring nodes in an EEC-NET with a tree-topology. This is achieved through a pre-trained lightweight decoder that generates synthetic samples (referred to as bridge samples), which are used to extract logits for knowledge distillation across neighboring nodes. SKR mitigates the propagation of misleading knowledge by refining the transferred predictions through a series of corrective steps. It employs knowledge queues to cache reliable historical predictions, applies maximum likelihood estimation to adjust class probabilities based on this cached knowledge, and uses relative entropy minimization to preserve the inter-class relationships of the corrected probabilities. We also demonstrate that FedEEC supports arbitrary non-cloud nodes switching between upper-tier servers, providing flexibility guarantees for knowledge agglomeration in multi-tier networks.

In summary, the main contributions of this paper are as follows:

- We propose a novel HFL framework suitable for EECC paradigm, named End-Edge-Cloud Federated Learning with Self-Rectified Knowledge Agglomeration (FedEEC), which recursively organizes computing nodes via a customized interaction protocol and enables models trained on the end, edge, and cloud nodes to grow larger in size and stronger in generalization ability.
- We introduce Bridge Sample Based Online Distillation Protocol (BSBODP) and Self-Knowledge Rectification (SKR) to facilitate model-agnostic collaborative training among neighboring nodes, and to refine the transferred class probabilities to mitigate the propagation of misleading predictions, respectively. Both mechanisms are theoretically grounded to support the dynamic migration of computing nodes in EECC environments.
- We validate the effectiveness of FedEEC in an end-edge-cloud architecture over SVHN, CIFAR-10, and CINIC-10 datasets. Empirical results demonstrate that FedEEC achieves better cloud model accuracy in all considered settings compared with state-of-the-art methods.

II. PRELIMINARY

A. End-Edge-Cloud Collaboration

End-Edge-Cloud Collaboration (EECC) is an emerging computing paradigm that coordinates multi-tier heterogeneous computing nodes including central cloud servers, numerous edge servers, and massively distributed end devices, aiming to work on large-scale computing tasks collaboratively. Typically, end devices are located at the periphery of the network, where they generate data and perform data-intensive AI applications. Edge servers bridge to connect end devices and cloud servers, distributed at intermediate locations along the network. Cloud servers, situated at the network's core, are equipped with sufficient computational and storage resources, facilitating the coordination of underlying end devices and edge servers.

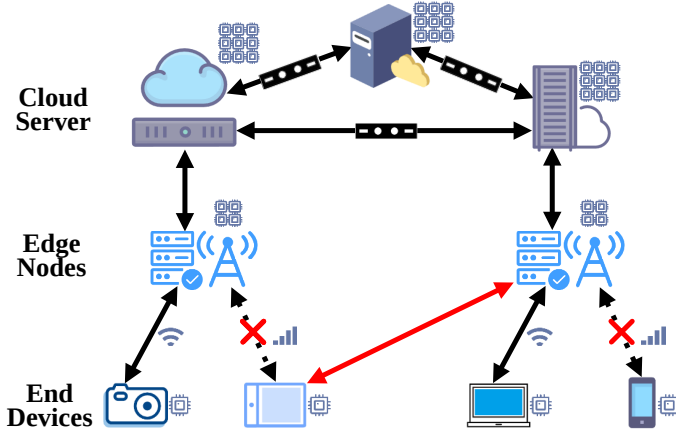


Fig. 1. Hierarchical topology of end-edge-cloud Collaboration with dynamic node migration.

In this paper, we consider a typical End-Edge-Cloud Network (EEC-NET) with a tree topology (shown in Fig. 1) denoted as $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of computing nodes and $\mathcal{E}$ is the set of communication links. The EEC-NET G is structured with a single root node $r \in \mathcal{V}$ and one or more leaf nodes, which form the set $\mathcal{L} \subset \mathcal{V}$. Each computing node $v \in \mathcal{V}$ except leaf nodes has one or more children, collectively forming a set $Child(v)$. Any computing node $v \in \mathcal{V}$ except the root node is associated with a single parent node, denoted as $Parent(v)$. Additionally, we define $Leaf(v)$ as the set of all leaf nodes within the sub-tree that seeks the computing node v as its root, and hence we have $Leaf(r) = \mathcal{L}$. For organization, the computing nodes in the EEC-NET are arranged into hierarchical tiers. Assuming the EEC-NET G have a total of T tiers, and the computing nodes in tier $t \in \{1, 2, \dots, T\}$ form a set $\mathcal{V}_t$, so that we have $\mathcal{V}_T = \mathcal{L}$ and $\mathcal{V}_1 = \{r\}$. For reasons of flexibility, EECC should take into account the dynamic migration of computing nodes, as illustrated in Fig. 1. Each non-root node in the EEC-NET should dynamically reestablish its parent node to efficiently redistribute workloads and recover from disruptions, ensuring uninterrupted operations and improving overall system functionality.

B. Hierarchical Federated Learning Empowered by End-Edge-Cloud Collaboration

We consider a scenario where $K = |\mathcal{L}|$ end devices (called clients) participate in Hierarchical Federated Learning (HFL) over an EEC-NET. Each client $k \in \{1, 2, \dots, K\}$ owns a local dataset $\mathcal{D}^k = \bigcup_{i=1}^{N^k} \{(X_i^k, y_i^k)\}$, in which N^k is the number of samples in $\mathcal{D}^k$, and X_i^k, y_i^k are the data and label of the i -th sample in $\mathcal{D}^k$, respectively. Besides, each node $v \in \mathcal{V}$ holds a model $Model(v)$ with parameters W^v , and can exchange information with its parent and child nodes, i.e., $\{Parent(v)\} \cup Child(v)$. Our goal is to obtain the model

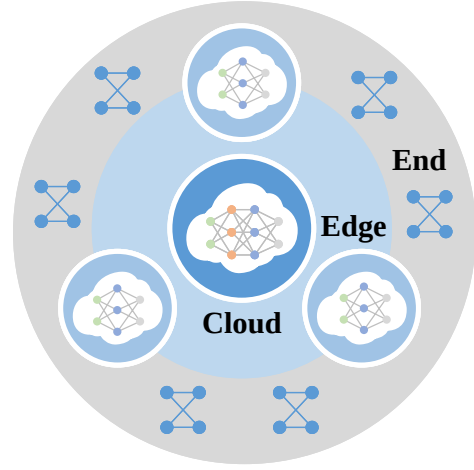


Fig. 2. Expected model scales distribution across tiers in an end-edge-cloud network.

in the root node $Model(r)$ that minimizes its training loss $L_{train}(\cdot)$ over all private data on devices, formulated as:

$$\underset{W^r}{\operatorname{argmin}} L_{train}\left(\bigcup_{k=1}^K \mathcal{D}^k; W^r\right). \quad (1)$$

To fully exploit heterogeneous resources and accommodate the differentiated capabilities of end, edge, and cloud computing nodes, it is crucial to guarantee edge and cloud servers, which possess more powerful computational resources, deploy larger models than resource-constrained end devices, as shown in Fig. 2. These larger models are more capable of extracting complex and generalized patterns from private data, thereby fully leveraging upper-tier resources to deliver substantial improvements in system performance.

C. Interaction Protocols in Federated Learning

Interaction protocols in FL define the rules for information exchange between computing nodes. Prevailing FL interaction protocols are based on parameters interaction like FedAvg [5], where upper-tier nodes aggregate and distribute model parameters between themselves and their subordinate nodes. Without considering child selection, this aggregation protocol typically keeps to the following steps to update the model on the non-root computing node v^* and its parent node $Parent(v^*)$:

- v^* uploads its model parameters W^{v^*} to $Parent(v^*)$.
- $Parent(v^*)$ waits for all its child nodes $Child(Parent(v^*))$ to finish uploading their model parameters.
- $Parent(v^*)$ aggregates the uploaded model parameters according to the following assignment to obtain updated model parameters:

$$W^{Parent(v^*)} \leftarrow \frac{\sum_{v \in Child(Parent(v^*))} W^v \cdot \sum_{u \in Leaf(v)} |\mathcal{D}^u|}{\sum_{v \in Child(Parent(v^*))} \sum_{u \in Leaf(v)} |\mathcal{D}^u|} \quad (2)$$

TABLE I
COMPARISON OF FEDEEC WITH EXISTING FL METHODS.

Method	End-Edge-Cloud Collaboration	Larger Model on the Cloud	Dynamic Migration	Training without Public Data
FedAvg [5]	✗	✗	✗	✓
FedProx [6]	✗	✗	✗	✓
FedGems [20]	✗	✓	✗	✗
FedET [21]	✗	✓	✗	✗
FedRolex [22]	✗	✓	✗	✓
FedGKT [23]	✗	✓	✗	✓
FedDKC [24]	✗	✓	✗	✓
DemLearn [25]	✓	✗	✗	✓
FedHKT [18]	✓	✓	✗	✗
HierFAVG [11]	✓	✗	✓	✓
FedCH [13]	✓	✗	✗	✓
HierMo [14]	✓	✗	✓	✓
FedAgg [26]	✓	✓	✓	✓
FedEEC	✓	✓	✓	✓

where $|\mathcal{D}^u|$ denotes the number of local data samples from the device corresponding to the leaf node u .

- $Parent(v^*)$ distributes the updated model parameters $W^{Parent(v^*)}$ to all its child nodes.
- v^* updates its model parameters with those downloaded from its parent node $Parent(v^*)$.

III. MOTIVATION

A. Knowledge Agglomeration

To exploit the computational power of edge and cloud nodes within the HFL process, we prompt the intuitive motivation of training larger models on these computing nodes than end devices for achieving higher accuracy with stronger generalization abilities. However, a key challenge arises: how can we effectively facilitate collaboration between models with different structures across the hierarchical tiers of an EEC-NET? Traditional parameters aggregation, as used in prevailing EECC-empowered FL methods [11], [14], [27], is inadequate because it requires uniform model structures to be adopted across all computing nodes, limiting the scale of trained models by the least powerful end devices and thus underutilizing the computational power of edge and cloud servers.

To overcome this challenge, we propose utilizing on-line distillation [28], [29] as an interaction protocol among computing nodes at different tiers, which enables different nodes to train models with heterogeneous structures via iteratively exchanging the logits (also called knowledge) between two neighboring nodes (also called parent-child node pairs), guiding reciprocal model training while supporting potential knowledge agglomeration propagating up the tree hierarchy toward the cloud computing node. However, directly integrating online distillation into EECC-powered FL may raise privacy concerns about sharing data on devices, as the same sample is required to extract logits across different computing nodes. Therefore, we propose to employ synthetic data as a bridge to transfer knowledge across multi-tier nodes with the assistance of a lightweight pre-trained autoencoder. This synthetic data is generated by the decoder using either encoded or received embeddings, thereby enabling the computation of

logits without revealing raw data on devices. Empowered by this interaction protocol, our hierarchical knowledge transfer framework allows the FL system to take full advantage of the heterogeneous computing capabilities across different tiers in EEC-NET, from resource-constrained end devices to more powerful edge and cloud nodes.

B. Knowledge Rectification

The bottom-up knowledge agglomeration process in EECC-empowered FL faces challenges in ensuring the reliability of transferred knowledge between neighboring nodes. Due to the resource constraints of end devices and the inherently scarce, non-IID nature of their data, models trained at these lower tiers may generate biased or misleading predictions. When erroneous knowledge propagates upwards to edge or cloud nodes, it amplifies errors and ultimately degrades the optimization of the cloud model.

To tackle this issue, we adaptively rectify knowledge within every computing node to mitigate the propagation of misleading patterns during interactions. Specifically, we leverage historical knowledge queues and statistical probabilistic adjustments to correct potentially misleading predictions, guaranteeing precise adjustment of softmax probabilities for label-relevant classes, while maintaining the similarity integrity for the remaining classes with theoretical support. This adaptive rectification mechanism not only improves the quality of transferred knowledge but also enhances the robustness of the entire EECC-empowered FL system against potential biases and errors introduced by individual nodes.

IV. END-EDGE-CLOUD FEDERATED LEARNING WITH SELF-RECTIFIED KNOWLEDGE AGGLOMERATION

A. Framework Overview

We introduce End-Edge-Cloud Federated Learning with Self-Rectified Knowledge Agglomeration (FedEEC), a novel EECC-empowered FL framework that supports tier-by-tier model scaling across end, edge, and cloud computing nodes within an EEC-NET. A detailed comparison of FedEEC with existing FL methods is provided in Table I. In FedEEC,

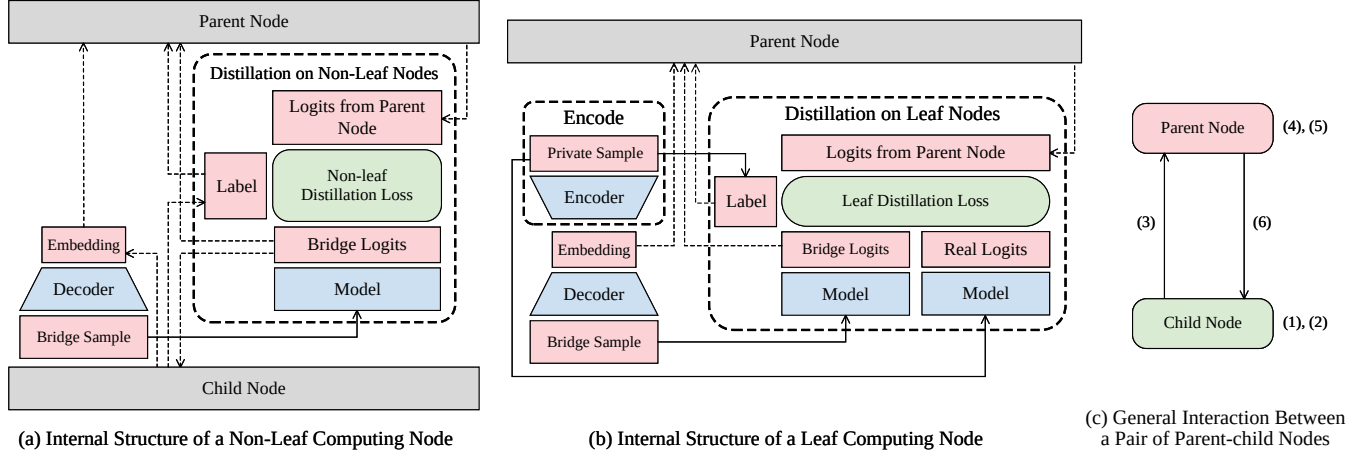


Fig. 3. Overview of BSBODP. (1) Child node distillation on bridge samples. (2) Logits extraction on child node. (3) Upload logits to parent node. (4) Parent node distillation on bridge samples. (5) Logits extraction on parent node. (6) Distribute logits to child node.

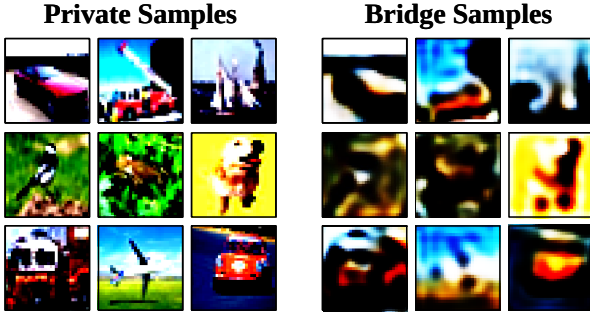


Fig. 4. Comparison of private samples and bridge samples.

each node can adopt a model structure that matches its own computing power, and the lower-tier nodes can iteratively transfer their learned knowledge to the upper-tier nodes via a customized interaction protocol. Therefore, the cloud model can ultimately integrate the knowledge from all nodes and capitalize on its inherently larger capacity to capture complex patterns, thereby delivering superior performance. To facilitate general interaction between computing nodes in FedEEC, we propose the Bridged Sample-Based Online Distillation Protocol (BSBODP), which ensures privacy and flexibility during multi-tier collaborative training by performing logit-based mutual knowledge distillation on generated synthetic samples. Furthermore, to enhance the quality of knowledge transfer and improve the robustness of the overall FL system, we design a Self-Knowledge Rectification (SKR) mechanism. Leveraging principles of maximum likelihood estimation and relative entropy minimization, SKR dynamically rectifies the transferred softmax probabilities based on reliable historical classification results, enhancing resistance to the propagation of misleading knowledge during interactions. By combining the advantages of both BSBODP and SKR, FedEEC recursively organizes computing nodes of each tier in EEC-NET, while complying with continuous growth in model scale and

generalization ability from the bottom to the top. Additionally, FedEEC can also handle scenarios where any non-root computing node dynamically changes its parents, which ensures deployment flexibility in a realistic EEC-NET. In the following subsections, we will provide detailed elaborations on the technical details of FedEEC and demonstrate its deployment flexibility within EEC-NET with theoretical support.

B. Bridge Sample Based Online Distillation Protocol

Fig. 3 provides the overview of our designed BSBODP, where each computing node is equipped with a pre-trained decoder and a conventional complete model (the model to be trained on the computing node). In particular, each leaf computing node additionally preserves a pre-trained encoder (forms a pre-trained autoencoder with the aforementioned decoder) for encoding local data into embeddings, which will be transmitted to its parent node until reaching the root node. During execution, each computing node utilizes the decoder to generate bridge samples that match the data distribution of its corresponding leaf nodes based on the embeddings encoded from private data or uploaded by sub-nodes. These bridge samples are used as intermediaries, facilitating online distillation between every pair of parent-child computing nodes in the tree hierarchy, with alignment ensured by the same embedding. As illustrated in Fig. 4, it is challenging to recover raw information from the embeddings of private samples since the embeddings are generated based on an extremely lightweight autoencoder ($<50K$ model parameters) that are pre-trained on a super large open dataset (such as ImageNet [30]). The autoencoder used to generate bridge samples is intentionally designed with limited capacity, making it hard to reconstruct fine-grained features of the original data. This makes online distillation a privacy-acceptable interaction protocol between neighboring computing nodes.

We formulate the process of BSBODP as follows: the decoder on each computing node is defined as $dec(\cdot)$, and the encoder on each leaf computing node is defined as $enc(\cdot)$.

Algorithm 1: Bridge Sample Based Online Distillation Protocol (BSBODP)

Input: v^1, v^2 **Output:** Trained $Model(v^1), Model(v^2)$ **procedure** BSBODP(v^1, v^2) BSBODP-DIRECTIONAL(v^1, v^2) BSBODP-DIRECTIONAL(v^2, v^1) **return** $Model(v^1), Model(v^2)$ **end procedure****procedure** BSBODP-Directional(v^S, v^T) v^T generates bridge samples $dec(\varepsilon)$ from all stored embeddings ε v^T extracts logits $z^\varepsilon = f(dec(\varepsilon); W^{v^T})$ on bridge samples, and transmits the results to v^S **if** $v^S \notin \mathcal{L}$ **then** Optimize W^{v^S} according to Eq. (3) **end** **else** Optimize W^{v^S} according to Eq. (5) **end****end procedure**

In addition, the model to be trained on any computing node $Model(v^*)$, $v^* \in \mathcal{V}$ can give an inference on the input data via $f(\cdot; W^{v^*})$. When performing upward knowledge distillation, the child node in any pair of parent-child computing nodes acts as the teacher v^T , and the parent node acts as the student v^S . When performing downward knowledge distillation, the roles are reversed. Specifically, each model on non-leaf student computing node $Model(v^S)$, $\forall v^S \notin \mathcal{L}$ distills the knowledge from the model on its teacher $Model(v^T)$ over the bridge samples that are generated from the embeddings ε corresponding to $\bigcup_{u \in Leaf(v^S)} \mathcal{D}^u \cap \bigcup_{u \in Leaf(v^T)} \mathcal{D}^u$. With generated bridge samples as intermediates, $Model(v^S)$ is optimized according to the non-leaf distillation loss as follows:

$$\begin{aligned}
 & \min_{W^{v^S}} L_{non-leaf} \\
 &= \min_{W^{v^S}} [L_{CE}(\tau(f(dec(\varepsilon); W^{v^S})), y^\varepsilon) + \\
 & \quad \beta \cdot KL(\tau(f(dec(\varepsilon); W^{v^S})) || \tau(\frac{z^\varepsilon}{T}))] \\
 &= \min_{W^{v^S}} [L_{CE}(\tau(f(dec(\varepsilon); W^{v^S})), y^\varepsilon) + \\
 & \quad \beta \cdot KL(\tau(f(dec(\varepsilon); W^{v^S})) || \underbrace{\tau(\frac{f(dec(\varepsilon); W^{v^T})}{T})}_{\frac{z^\varepsilon}{T}})], \tag{3}
 \end{aligned}$$

where $L_{CE}(\cdot)$ is the cross-entropy loss function, $KL(\cdot)$ is the Kullback-Leibler divergence loss function, and β is the distillation weight. Moreover, y^ε and z^ε are the labels and extracted logits of bridge samples corresponding to embeddings ε uploaded from child nodes, and ε is ultimately generated at the leaf nodes according to the following equation:

$$\varepsilon = enc(X^*), \forall (X^*, y^*) \in \bigcup_{u \in Leaf(v^S)} \mathcal{D}^u \cap \bigcup_{u \in Leaf(v^T)} \mathcal{D}^u. \tag{4}$$

Moreover, each model on leaf student computing node $Model(v^S)$, $\forall v^S \in \mathcal{L}$ is optimized subject to a linear com-

bination of the non-leaf distillation loss over generated bridge samples and the local training losses (controlled by a weight γ) over private samples $(X^*, y^*) \in \mathcal{D}^{v^S}$, that is:

$$\begin{aligned}
 & \min_{W^{v^S}} L_{leaf} \\
 &= \min_{W^{v^S}} L_{CE}(f(X^*; W^{v^S}), y^*) + \gamma \cdot L_{non-leaf}, \tag{5}
 \end{aligned}$$

where

$$\varepsilon = enc(X^*), \forall (X^*, y^*) \in \mathcal{D}^{v^S}. \tag{6}$$

Once the above steps are completed, v^S and v^T need to swap their roles and optimize in the opposite direction following the above constraints, as shown in Algorithm 1. Thereout, the models to be trained on every pair of parent-child computing nodes can learn from each other, and the representation learned through knowledge distillation can be propagated to the model on the cloud tier by tier, starting from the leaf nodes.

C. Self-Knowledge Rectification

To ensure reliable knowledge transfer in BSBODP, we propose Self-Knowledge Rectification (SKR), a mechanism designed to adaptively rectify misleading attributions in bridge sample knowledge caused by resource limitations and non-IID data at lower-tier nodes, preventing error propagation and enhancing system performance. SKR introduces a rectification process at each computing node, evaluating the predicted probabilities to identify potential inaccuracies and correcting them before transferring them to neighboring nodes. This process ensures that the transferred knowledge is more reliable and aligned with well-attributed historical predictions. In the following subsections, we detail the procedure of SKR and demonstrate how it systematically enhances hierarchical knowledge transfer within the EEC-NET.

1) *Misleading Class Probabilities Identification:* Consider an arbitrary computing node $v^* \in \mathcal{V}$ in the EEC-NET, we consider the cases where an arbitrary bridge sample $dec(\varepsilon)$ belonging to class $c = y^*$ is used to extract logits z^ε . For a specific $dec(\varepsilon)$, the original probability estimation for class c is derived from the softmax-processed logits, that is:

$$p_c = \tau(\frac{z^\varepsilon}{T})_c = \tau(\frac{f(dec(\varepsilon); W^{v^*})}{T})_c, \tag{7}$$

where p_c is the predicted probability of classifying $dec(\varepsilon)$ as class c . However, the model's prediction confidence may be misplaced, resulting in the estimated c -class probability p_c being incorrectly attributed. Specifically, for a bridge sample of class c , the model might assign higher probability estimates to one or more non- c classes than to class c , that is:

$$p_c < \tau(\frac{z^\varepsilon}{T})_i, \exists i \in \{1, 2, \dots, C\} \wedge i \neq c. \tag{8}$$

This condition highlights scenarios where the class predictions are incorrectly attributed, which can introduce biased updates that propagate through the EEC-NET hierarchy, ultimately degrading the performance of the cloud-based model.

2) *Knowledge Queues*: To assist the knowledge rectification process, we introduce a knowledge queue to store historical class probabilities from instances where the model on v^* correctly estimates c -class bridge samples. The knowledge queue for class c can hold up to B elements and is denoted as $\mathcal{B}_c = [q_c^1, q_c^2, \dots, q_c^B]$, where each element in the queue $q_c^j \in \mathcal{B}_c, \forall j \in \{1, 2, \dots, B\}$ records the value of p_c from instances where the model correctly identified any c -class bridge sample $dec(\varepsilon)$ as belonging to class c in previous inference processes. When the queue reaches its capacity, the addition of new elements results in the removal of the oldest elements, ensuring that $\mathcal{B}_c$ always contains the most recent correctly attributed knowledge.

3) *Knowledge Rectification for Label-relevant Classes*: To rectify knowledge of label-relevant classes, we utilize historical knowledge stores in knowledge queues and apply maximum likelihood estimation to compute rectified probability estimates p'_c across bridge samples $dec(\varepsilon)$ belonging to class c . Assuming the historical well-attributed probabilities $\{q_c^1, q_c^2, \dots, q_c^B\}$ are independently and identically distributed, following a certain probability distribution $P(\mathcal{B}_c|p'_c)$. We define the likelihood function $L_{p'_c}$ as the joint probability of $\{q_c^1, q_c^2, \dots, q_c^B\}$ conditioned on the parameter p'_c , that is:

$$L_{p'_c} = P(q_c^1, q_c^2, \dots, q_c^B | p'_c). \quad (9)$$

Given that $\{q_c^1, q_c^2, \dots, q_c^B\}$ are mutually independent, we can derive:

$$L_{p'_c} = P(q_c^1, q_c^2, \dots, q_c^B | p'_c) = \prod_{i=1}^B P(q_c^i | p'_c). \quad (10)$$

Therefore, the log-likelihood function $\ell(p'_c)$ can be formulated as:

$$\ell_{p'_c} = \log L_{p'_c} = \log \left(\prod_{i=1}^B P(q_c^i | p'_c) \right) = \sum_{i=1}^B \log P(q_c^i | p'_c). \quad (11)$$

Assuming $q_c^i \in \mathcal{B}_c, \forall i \in \{1, 2, \dots, B\}$ follows a Gaussian distribution with mean p'_c and variance s^2 , i.e., $q_c^i \sim \mathcal{N}(p'_c, s^2), \forall i \in \{1, 2, \dots, B\}$. Then, the probability distribution $P(q_c^i | p'_c)$ can be formulated as:

$$P(q_c^i | p'_c) = \frac{1}{\sqrt{2\pi}s} \exp \left(-\frac{(q_c^i - p'_c)^2}{2s^2} \right). \quad (12)$$

Substituting Eq. (12) into Eq. (13), we have:

$$\begin{aligned} \ell_{p'_c} &= \sum_{i=1}^B \log \left(\frac{1}{\sqrt{2\pi}s} \exp \left(-\frac{(q_c^i - p'_c)^2}{2s^2} \right) \right) \\ &= \sum_{i=1}^B \left(\log \frac{1}{\sqrt{2\pi}s} - \frac{(q_c^i - p'_c)^2}{2s^2} \right) \\ &= B \log \frac{1}{\sqrt{2\pi}s} - \frac{1}{2s^2} \sum_{i=1}^B (q_c^i - p'_c)^2. \end{aligned} \quad (13)$$

To maximize the log-likelihood function $\ell(p'_c)$, we compute the derivative with respect to p'_c in Eq. (13) and set it to zero, which means:

$$\frac{d\ell_{p'_c}}{dp'_c} = -\frac{1}{2s^2} \sum_{i=1}^B 2(q_c^i - p'_c)(-1) = \frac{1}{s^2} \sum_{i=1}^B (q_c^i - p'_c) = 0. \quad (14)$$

Solving the above equation, we can derive the result of p'_c , that is:

$$p'_c = \frac{1}{B} \sum_{i=1}^B q_c^i. \quad (15)$$

4) *Knowledge Rectification for Non-label Classes*: To rectify knowledge of non-label classes, we aim to preserve their relative relationships while accommodating the rectified probability of label-relevant classes. Specifically, we reconstruct the transferred estimated probabilities to $\mathcal{Q} = [p'_1, p'_2, \dots, p'_C]$, ensuring that the original predicted probabilities $\mathcal{P} = [p_1, p_2, \dots, p_C]$ have the smallest relative entropy (Kullback-Leibler divergence) with $\mathcal{Q}$, so as to preserve the original class relevance information as much as possible. The expanded form of relative entropy between $\mathcal{P}$ and $\mathcal{Q}$ is as follows:

$$KL(\mathcal{P} \parallel \mathcal{Q}) = \sum_{i=1}^C p_i \log \frac{p_i}{p'_i}. \quad (16)$$

We aim to minimize the relative entropy between $\mathcal{P}$ and $\mathcal{Q}$ under the premise of Eq. (15) and the basic properties of probability, which means:

$$\min_{p'_i} KL(\mathcal{P} \parallel \mathcal{Q}), \quad (17)$$

subject to:

$$\sum_{i=1}^C p'_i = 1, \quad (18)$$

$$p'_i \geq 0, \forall i \in \{1, 2, \dots, C\}, \quad (19)$$

$$p'_c = \frac{1}{B} \sum_{i=1}^B q_c^i. \quad (20)$$

We formulate the Lagrangian function L_λ with multiplier λ to handle the relative entropy between $\mathcal{P}$ and $\mathcal{Q}$ and related constraints, that is:

$$L_\lambda(p'_1, \dots, p'_C, \lambda) = \sum_{i=1}^C p_i \log \frac{p_i}{p'_i} + \lambda \left(\sum_{i=1}^C p'_i - 1 \right). \quad (21)$$

When $i \neq c$, we take the partial derivative with respect to p'_i in Eq. (21), that is:

$$\frac{\partial L_\lambda}{\partial p'_i} = \frac{\partial}{\partial p'_i} \left(p_i \log \frac{p_i}{p'_i} \right) + \frac{\partial}{\partial p'_i} \left(\lambda \sum_{j=1}^C p'_j - \lambda \right). \quad (22)$$

Compute the partial derivative of the first part, we have:

$$\frac{\partial}{\partial p'_i} \left(p_i \log \frac{p_i}{p'_i} \right) = p_i \frac{\partial}{\partial p'_i} (\log p_i - \log p'_i) = -\frac{p_i}{p'_i}. \quad (23)$$

Compute the partial derivative of the second part, we have:

$$\frac{\partial}{\partial p'_i} \left(\lambda \sum_{j=1}^C p'_j - \lambda \right) = \lambda. \quad (24)$$

Combining the two parts mentioned in Eq. (23) and Eq. (24), we have:

$$\frac{\partial L_\lambda}{\partial p'_i} = -\frac{p_i}{p'_i} + \lambda. \quad (25)$$

Setting the combined partial derivative to zero, we obtain:

$$\frac{\partial L_\lambda}{\partial p'_i} = -\frac{p_i}{p'_i} + \lambda = 0. \quad (26)$$

Hence,

$$p'_i = \frac{p_i}{\lambda} \quad (i \neq c). \quad (27)$$

Based on the derived result in Eq. (27), we leverage the known p'_c in Eq. (15) and the probability sum constraint in Eq. (18) to determine λ as follows:

$$p'_c + \sum_{i=1, i \neq c}^C \frac{p_i}{\lambda} = 1. \quad (28)$$

Thus,

$$\lambda = \frac{\sum_{i=1, i \neq c}^C p_i}{1 - p'_c}. \quad (29)$$

We substitute the value of λ in Eq. (27) to compute p'_i when $i \neq c$, that is:

$$p'_i = \frac{p_i}{\lambda} = \frac{p_i(1 - p'_c)}{\sum_{i=1, i \neq c}^C p_i} = \frac{p_i(1 - \frac{1}{B} \sum_{i=1}^B q_c^i)}{\sum_{i=1, i \neq c}^C p_i}. \quad (30)$$

Obviously, the derived results of p'_i ($i \neq c$) satisfy the non-negativity constraint in Eq. (19). At this point, we can conclude that:

$$p'_i = \begin{cases} \frac{p_i(1 - \frac{1}{B} \sum_{i=1}^B q_c^i)}{\sum_{i=1, i \neq c}^C p_i}, & \text{if } i \neq c \\ \frac{1}{B} \sum_{i=1}^B q_c^i, & \text{if } i = c \end{cases} \quad (31)$$

This entire process ensures that the rectified knowledge $\mathcal{Q} = [p'_1, p'_2, \dots, p'_C]$ transferred by v^* is both accurate and aligned with historical predictions.

5) *Application of SKR in BSBODP*: To apply SKR in BSBODP, we consider a scenario where v^* serves as the teacher computing node v^T , with its corresponding neighboring node acts as the student computing node v^S . For any node pairs v^T and v^S , the rectified knowledge $\mathcal{Q}$ generated by v^T is used to guide the training of v^S . In this case, the optimization objective for the non-leaf v^S can be reformulated as:

$$\begin{aligned} & \min_{W^{v^S}} L'_{non-leaf} \\ & = \min_{W^{v^S}} [L_{CE}(\tau(f(dec(\varepsilon); W^{v^S})), y^\varepsilon) + \\ & \quad \beta \cdot KL(\tau(f(dec(\varepsilon); W^{v^S})) || \mathcal{Q})]. \end{aligned} \quad (32)$$

Correspondingly, the optimization objective of leaf v^S is re-represented as:

$$\begin{aligned} & \min_{W^{v^S}} L'_{leaf} \\ & = \min_{W^{v^S}} L_{CE}(f(X^*; W^{v^S}), y^*) + \gamma \cdot L'_{non-leaf}. \end{aligned} \quad (33)$$

Algorithm 2: Bridge Sample Based Online Distillation Protocol (BSBODP) with Self-Knowledge Rectification (SKR)

Input: v^1, v^2

Output: Trained $Model(v^1), Model(v^2)$

procedure BSBODP-SKR(v^1, v^2)

 BSBODP-SKR-DIRECTIONAL(v^1, v^2)

 BSBODP-SKR-DIRECTIONAL(v^2, v^1)

return $Model(v^1), Model(v^2)$

end procedure

procedure BSBODP-SKR-Directional(v^S, v^T)

v^T generates bridge samples $dec(\varepsilon)$ from all stored embeddings ε

v^T extracts logits $z^\varepsilon = f(dec(\varepsilon); W^{v^T})$ on bridge samples

v^T computes softmax probabilities $\mathcal{P} = \tau(\frac{z^\varepsilon}{T})$

if $p_{y^*} < \tau(\frac{z^\varepsilon}{T})_i, \exists i \in \{1, 2, \dots, C\}$ **then**

if $|\mathcal{B}_{y^*}| = 0$ **then**

 Transmits $\mathcal{P}$ to v^S

end

else

 Compute $\mathcal{Q}$ according to Eq. (31)

 Transmits $\mathcal{Q}$ to v^S

end

else

if $|\mathcal{B}_{y^*}| = B$ **then**

$\mathcal{B}_{y^*}.pop()$

end

$\mathcal{B}_{y^*}.push(p_{y^*})$

 Transmits $\mathcal{P}$ to v^S

end

if Receive $\mathcal{P}$ from v^T **then**

if $v^S \notin \mathcal{L}$ **then**

 Optimize W^{v^S} according to Eq. (3)

end

else

 Optimize W^{v^S} according to Eq. (5)

end

end

else

if $v^S \notin \mathcal{L}$ **then**

 Optimize W^{v^S} according to Eq. (32)

end

else

 Optimize W^{v^S} according to Eq. (33)

end

end

end procedure

The execution procedure of BSBODP combined with SKR is listed in Algorithm 2. Therefore, the transferred knowledge can be adaptively rectified within each computing node before it is propagated to other nodes, improving the robustness of the entire EECC-empowered FL system.

D. Recursive Agglomeration in End-Edge-Cloud Networks

Considering FL in an EEC-NET where computing nodes are organized in a tree topology, we propose the FedEEC framework. Through the combination of BSBODP and SKR, FedEEC enables collaborative training over every pair of parent-child computing nodes, recursively distilling knowledge

Algorithm 3: End-Edge-Cloud Federated Learning with Self-Rectified Knowledge Agglomeration (FedEEC)

Input: $\mathcal{V}, \mathcal{E}$
Output: Trained $Model(v^*), \forall v^* \in \mathcal{V}$

```

procedure FedEEC( $\mathcal{V}, \mathcal{E}$ )
  INIT( $r, \mathcal{E}$ )
  while do not reach maximum communication rounds
  do
    FEDEECTRAIN( $r, \mathcal{E}$ )
  end
end procedure
procedure Init( $v^*, \mathcal{E}$ )
  if  $v^* = r$  then
    for  $u \in Child(v^*)$  in parallel do
      INIT( $u, \mathcal{E}$ )
      Receive and store embeddings  $\varepsilon$  with
      corresponding labels from  $u$ 
    end
  else if  $v^* \in \mathcal{L}$  then
    Extract and store embeddings  $\varepsilon$  via
     $enc(X^*), \forall X^* \in \mathcal{D}^{v^*}$ 
    Transmit embeddings  $\varepsilon$  with corresponding
    labels to  $Parent(v^*)$ 
  end
  else
    for  $u \in Child(v^*)$  in parallel do
      INIT( $u, \mathcal{E}$ )
      Receive and store embeddings  $\varepsilon$  from  $u$ 
      Send embeddings  $\varepsilon$  with corresponding
      labels to  $Parent(v^*)$ 
    end
  end
end procedure
procedure FedEECTrain( $v^*, \mathcal{E}$ )
  if  $v^* = r$  then
    for  $u \in Child(v^*)$  in parallel do
      FEDEECTRAIN( $u, \mathcal{E}$ )
    end
  else if  $v^* \in \mathcal{L}$  then
    BSBODP-SKR( $v^*, Parent(v^*)$ )
  end
  else
    for  $u \in Child(v^*)$  in parallel do
      FEDEECTRAIN( $u, \mathcal{E}$ )
      BSBODP-SKR( $v^*, Parent(v^*)$ )
    end
  end
end procedure

```

from bottom to top in an agglomerative manner, as formulated in Algorithm 3. Specifically, FedEEC includes two main phases: initialization and recursive training with BSBODP and SKR. In the initialization phase, leaf nodes generate embeddings using their pre-trained encoders and send the embeddings up the tree hierarchy to the root node. In the training phase, each node recursively distills the knowledge from the sub-tree that seeks it as the root and passes the newly extracted knowledge to its parent, applying BSBODP combined with SKR to enable interaction between parent-child pairs. This process starts from the leaf nodes and ends with the cloud server, where the model on the cloud is updated with the learned knowledge from all tiers. The hierarchical structure of EEC-NET, combined with the model-agnostic nature of BSBODP and SKR, allows upper-tier nodes to deploy larger

models and integrate more knowledge from lower-tier nodes to achieve superior performance, and the largest model (i.e. the model on the cloud) is eventually the model that we want to gain through collaborative training.

E. Deployment Flexibility Guarantees in End-Edge-Cloud Network

FedEEC supports dynamic migration of computing nodes to guarantee deployment flexibility in an EEC-NET, where various factors such as mobility or unstable connections may require computing nodes to switch to a different parent. To demonstrate the advantage of FedEEC in handling dynamic migration of computing nodes, we classify existing FL interaction protocols that potentially support hierarchical collaborative model training into two types based on their constraints on model structures of the interacting computing nodes: equivalence interaction protocols and partial order interaction protocols. In the following part, we will give formal definitions of these two types of interaction protocols, and prove that equivalence interaction protocols, including our proposed BSBODP combined with SKR, provide better support for dynamic migration of computing nodes.

Definition 1. (Equivalence Interaction Protocol). Consider an EEC-NET with a tree topology formulated in section II-A, we define a binary relation R over any pair of parent-child computing nodes. The sufficient necessary condition for an equivalence interaction protocol is defined as follows:

$$< Model(v^i), Model(v^j) > \in R, \forall v^i \in \mathcal{V}, \quad (34)$$

$$\begin{aligned} < Model(v^i), Model(v^j) > \in R \wedge v^i \neq v^j \\ \rightarrow < Model(v^j), Model(v^i) > \in R, \forall v^i, v^j \in \mathcal{V}, \end{aligned} \quad (35)$$

$$\begin{aligned} < Model(v^i), Model(v^j) > \in R \\ \wedge < Model(v^j), Model(v^k) > \in R \\ \rightarrow < Model(v^i), Model(v^k) > \in R, \forall v^i, v^j, v^k \in \mathcal{V}, \end{aligned} \quad (36)$$

that is:

$$Model(v^i) \sim Model(v^j), \forall < v^i, v^j > \in \mathcal{E}. \quad (37)$$

According to our definition, we can easily prove that the followings are equivalent interaction protocols: 1) the same model structure is adopted on the parent and child computing nodes, i.e. $Model(v^i) = Model(v^j), \forall < v^i, v^j > \in \mathcal{E}$, which is represented by FedAvg [5]; 2) model-agnostic interaction protocols that do not impose restrictions on the model structures of parent and child computing nodes, i.e. $Model(v^i) \perp Model(v^j), \forall < v^i, v^j > \in \mathcal{E}$, which is represented by BSBODP combined with SKR.

Definition 2. (Partial Order Interaction Protocol). Consider an EEC-NET with a tree topology formulated in section II-A, we define a binary relation R over any pair of parent-child computing nodes. The sufficient necessary condition of a partial order interaction protocol is defined as follows:

$$< Model(v^i), Model(v^j) > \in R, \forall v^i \in \mathcal{V}, \quad (38)$$

$$\begin{aligned} < Model(v^i), Model(v^j) > \in R \\ \wedge < Model(v^j), Model(v^i) > \in R \\ \rightarrow v^i = v^j, \forall v^i, v^j \in \mathcal{V}, \end{aligned} \quad (39)$$

$$\begin{aligned}
& \langle \text{Model}(v^i), \text{Model}(v^j) \rangle \in R \\
& \wedge \langle \text{Model}(v^j), \text{Model}(v^k) \rangle \in R \\
& \rightarrow \langle \text{Model}(v^i), \text{Model}(v^k) \rangle \in R, \forall v^i, v^j, v^k \in \mathcal{V},
\end{aligned} \tag{40}$$

that is:

$$\text{Model}(v^i) \preceq \text{Model}(v^j), \forall \langle v^i, v^j \rangle \in \mathcal{E}. \tag{41}$$

We can also prove that partial training-based interaction protocols [22], [31], which require the model on the child node to be a sub-model of that on the parent node, i.e. $\text{Model}(v^i) \subseteq \text{Model}(\text{Parent}(v^i))$, $\forall \langle v^i, \text{Parent}(v^i) \rangle \in \mathcal{E}$ are partial order interaction protocols.

Theorem 1. *HFL methods based on equivalence interaction protocols allow the parent of any v^1 switch to $\text{Parent}(v^2)$ while preserving both operational integrity and interaction consistency between computing nodes in the EEC-NET, where v^1 and v^2 are non-root nodes, i.e. $\text{Model}(v^1) \sim \text{Model}(\text{Parent}(v^2))$, $\forall v^1, v^2 \in \mathcal{V} - \mathcal{V}_1$.*

Proof.

First, we prove that any two computing nodes in the EEC-NET satisfy the equivalence relation determined by R . Without loss of generality, we have two arbitrary non-root nodes $v^x, v^y \in \mathcal{V} - \mathcal{V}_1$ in the EEC-NET. Since $\mathcal{V}$ is a tree topology, any two nodes are connected. Therefore, there must exist a finite sequence of m adjacent intermediate nodes $v_1^*, v_2^*, \dots, v_m^*$ such that v^x and v^y are connected through them, that is:

$$\text{Model}(v^x) \sim \text{Model}(v_1^*), \tag{42}$$

$$\text{Model}(v^y) \sim \text{Model}(v_m^*), \tag{43}$$

$$\text{Model}(v_{i-1}^*) \sim \text{Model}(v_i^*), \forall i \in \{2, 3, \dots, m\}. \tag{44}$$

Due to the reflexivity and transitivity properties of equivalence relations, we can infer that:

$$\begin{aligned}
& \text{Model}(v^x) \sim \text{Model}(v_1^*) \sim \text{Model}(v_2^*) \\
& \sim \dots \sim \text{Model}(v_m^*) \sim \text{Model}(v^y).
\end{aligned} \tag{45}$$

Thus, we can conclude that:

$$\text{Model}(v^x) \sim \text{Model}(v^y). \tag{46}$$

Next, we prove that v^1 can change its parent to $\text{Parent}(v^2)$ while maintaining network integrity. Since both v^1 and $\text{Parent}(v^2)$ are distributed in $\mathcal{V}$, they satisfy an equivalence relation according to the aforementioned proof. Hence,

$$\text{Model}(v^1) \sim \text{Model}(\text{Parent}(v^2)). \tag{47}$$

Therefore, the computing node v^1 is allowed to become a child of $\text{Parent}(v^2)$, i.e. dynamic migration of computing nodes is allowed. $\square$

Theorem 2. *HFL methods based on partial order interaction protocols do not necessarily allow the parent of any v^1 switch to $\text{Parent}(v^2)$ while preserving both operational integrity and interaction consistency between computing nodes in the EEC-NET, where v^1 and v^2 are non-root nodes, i.e. $\neg \text{Model}(v^1) \preceq \text{Model}(\text{Parent}(v^2))$, $\exists v^1, v^2 \in \mathcal{V} - \mathcal{V}_1$.*

Proof.

We demonstrate that when v^1 and v^2 are at the same tier, partial order interaction protocols already no longer necessarily

allow parent switching for v^1 while maintaining satisfaction.

Case 2.1. When $v^1, v^2 \in \mathcal{V}_2$, we have:

$$\text{Parent}(v^1) = \text{Parent}(v^2) = r. \tag{48}$$

Hence, v^1 and v^2 have the same parent node r , and there is no dynamic migration of computing nodes in this case.

Case 2.2. When $v^1, v^2 \in \mathcal{V}_t, t \geq 3$, there are two sub-cases:

$$\text{Model}(\text{Parent}(v^1)) \preceq \text{Model}(\text{Parent}(v^2)), \tag{49}$$

and

$$\text{Model}(\text{Parent}(v^2)) \preceq \text{Model}(\text{Parent}(v^1)). \tag{50}$$

When Eq. (50) is satisfied, there exists a situation where computing node v^1 is not allowed to switch its parent to $\text{Parent}(v^2)$. Instantiating $\preceq$ to the partial order relation over integers $\leq$, we construct a tree topology $10(9(8, 7), 5(4, 3))$ and set function $\text{Model}(\cdot)$ to be a constant function, i.e. $\text{Model}(x) = x, \forall x$. In our setting, the parent of 7 (v^1) and 3 (v^2) are at the same tier and $\text{Model}(\text{Parent}(3)) = 5 \leq 9 = \text{Model}(\text{Parent}(7))$ satisfies Eq. (50). At this point, $\neg \text{Model}(7) \leq \text{Model}(\text{Parent}(3))$ is equivalent to $\neg 7 \leq 5$, which is apparently true. Hence, dynamic migration of computing nodes is not necessarily allowed. $\square$

V. EXPERIMENTS

A. Experiment Setup

1) *Hardware and Software Platforms:* We conduct experiments on a Huawei 2288H V5 physical server equipped with two Intel(R) Xeon(R) Gold 5218 CPUs, two NVIDIA RTX 3090 GPUs, and 384 GB AM. To comprehensively evaluate the performance of our proposed FedEEC, we adopt the FedML research library [32] deployed on Linux 6.8.0-38-generic, as the software platform for all experiments.

2) *Basic Settings:* We select three representative datasets, SVHN [33], CIFAR-10 [34], and CINIC-10 [35] to assess the performance of FedEEC under varying data scenarios. The primary evaluation metric used in this paper is cloud model accuracy. Regarding model structures, we deploy four types of models for collaborative training, which are M_{end}^1 , M_{end}^2 , M_{edge} , and M_{cloud} . Specifically, M_{end}^1 and M_{end}^2 are lightweight, three-layer Convolutional Neural Networks (CNNs) differing in the size of their intermediate layers. M_{edge} and M_{cloud} are Deep Residual Networks (ResNet) [36], designed with varying depths to suit the computational capabilities of edge and cloud nodes. Additionally, we introduce a six-layer autoencoder, M_{auto} for auxiliary training. M_{auto} comprises a three-layer encoder (M_{enc}) and a three-layer decoder (M_{dec}), pre-trained on ImageNet [30] dataset. The main configurations of adopted models are provided in Table II. To ensure fair comparisons, we control the key experimental configurations to isolate the impact of the HFL algorithms. The number of devices, data distributions, learning rate, and batch size are kept consistent across the same group of experiments to minimize the influence of non-algorithmic factors.

3) *Baseline Algorithms*: We compare FedEEC against the following five HFL or distributed learning algorithms:

- HierFAVG [11], which is a classical client-edge-cloud FL framework that introduces edge servers for partial model aggregation, reducing communication overhead and shortening training time.
- DemLearn [25], which is a distributed learning method for complex learning tasks with adaptive hierarchical organization.
- HierMo [14], which is an HFL method incorporating momentum acceleration, which accelerates model convergence by aggregating momentum across clients and edge servers.
- HierQSGD [15], which is an HFL method that integrates model quantization techniques to reduce communication overhead.
- FedAgg [26], which is the previous generation version of FedEEC, allowing trained models from end, edge to cloud nodes to grow larger in size and complexity through model-agnostic multi-tier collaboration.

B. Implementation Details

1) *Simulating Non-IID Data Distribution*: To simulate real-world non-independent and identically distributed (non-IID) on-device data, we utilize the data partition module in FedML to create heterogeneous data distributions. The degree of data heterogeneity is controlled by the hyperparameter $\alpha = 2.0$, which corresponds to the concentration parameter of the Dirichlet distribution used for partitioning data among devices [37]. For each experiment, we record the cloud model accuracy on the test dataset over 100 communication rounds, using the maximum accuracy observed during training as the final performance metric.

2) *Scaling Across Varying Number of Devices*: To evaluate the scalability of FedEEC with varying numbers of devices, we conduct experiments over 50, 100, and 500 devices. For each configuration, we set up 5, 10, and 20 edge servers, respectively, with devices evenly distributed across these servers. In the exceptional case of DemLearn, the number of edge servers is determined by the algorithm's adaptive organization process.

3) *Hierarchical Model Deployment*: For multi-tier aggregation algorithms (HierFAVG, DemLearn, HierMo, and HierQSGD), identical model structures are required across all computing nodes. Given the limited computational power of end devices, we uniformly deploy M_{end}^1 on end, edge, and cloud nodes for these algorithms. In contrast, for FedAgg and FedEEC that do not require model aggregation, we deploy M_{end}^1 , M_{edge} , and M_{cloud} at end, edge, and cloud nodes, respectively, to accommodate different computational capabilities across tiers. Additionally, we deploy the encoder (M_{enc}) on the end devices and distribute the decoder (M_{dec}) across all nodes to assist in the training process of FedAgg and FedEEC.

4) *Baseline Comparisons*: We conduct comprehensive evaluations of FedEEC against all considered baseline algorithms over SVHN and CIFAR-10 datasets. For CINIC-10 dataset, we compare FedEEC with HierFAVG, HierMo, HierQSGD, and FedAgg. Due to the significant computational overhead

TABLE II
MODEL CONFIGURATIONS USED IN EXPERIMENTS.

Model	Notation	Deployment	Parameters
CNN-1	M_{end}^1	End, Edge, Cloud	12.84K
CNN-2	M_{end}^2	End, Edge, Cloud	11.67K
ResNet-10	M_{edge}	Edge, Cloud	4.68M
ResNet-18	M_{cloud}	Cloud	10.66M
Encoder	M_{enc}	End	1.90K
Decoder	M_{dec}	End, Edge, Cloud	2.47K

introduced by the adaptive organization process in DemLearn, it becomes impractical to apply this method to such a large-scale dataset.

5) *Hyperparameters*: In all experiments, we set the learning rate to 0.001 and the batch size to 8. The specific hyperparameter configurations for each algorithm are as follows:

- For HierFAVG, we set $\kappa_1 = 1$ and $\kappa_2 = 1$.
- For DemLearn, we set $K_Levels = 2$ to form a three-tier end-edge-cloud architecture. Other hyperparameters follow the default setting in [38].
- For HierQSGD, we set $\kappa_1 = 1$, $\kappa_2 = 1$, $q_de = 1.0$, $q_ec = 1.0$. Other hyperparameters follow the default setting in [39].
- For HierMo, we set $\tau = 1$, $\pi = 1$, $\gamma = 0.5$ and $\gamma_a = 0.9$.
- For FedAgg, we set $\gamma = 1.0$, $T = 0.5$, and $\beta = 1.5$.
- For FedEEC, we set $B = 20$, $\gamma = 1.0$, $T = 0.5$, and $\beta = 1.5$.

C. Evaluation Results

1) *Performance Across Different Datasets*: Table III presents the cloud model accuracy achieved by FedEEC across various client numbers on three datasets. As shown, FedEEC consistently demonstrates strong performance across all datasets, outperforming the baseline methods in every case. On the SVHN dataset, FedEEC achieves an average accuracy of 80.72%, significantly exceeding the performances of HierFAVG (22.78%), DemLearn (27.10%), HierMo (19.81%), HierQSGD (19.59%), and FedAgg (76.75%). Similarly, FedEEC achieves average accuracies of 34.8% and 17.52% on the more challenging CIFAR-10 and CINIC-10 datasets, significantly outperforming the best baseline methods, which achieve 33.31% and 16.60% respectively. These results highlight FedEEC's consistent state-of-the-art performance across different datasets, demonstrating its robustness and adaptability to diverse data scenarios.

2) *Performance Across Different Number of Clients*: FedEEC exhibits robust performance across varying client number settings. As shown in Table III, FedEEC consistently outperforms the baseline methods across all datasets regardless of the number of clients. Notably, within the same dataset, FedEEC maintains stable cloud model accuracy, with fluctuations less than 2%. This robustness across varying client numbers underscores FedEEC's ability to effectively manage knowledge transfer and collaboration across tiers, even as data is distributed over a larger number of devices. The superior performance of FedEEC across varying client numbers demonstrates its reliability and scalability as a solution for FEL.

TABLE III
COMPARISON OF CLOUD MODEL ACCURACY (%) ACROSS DIFFERENT DATASETS AND CLIENT NUMBERS. **BOLD** VALUES REPRESENT THE BEST ACCURACY, THE SAME AS BELOW.

	Method	Trained Model			50 Clients	100 Clients	500 Clients	Average
		End	Edge	Cloud				
SVHN	HierFAVG	M_{end}^1			19.59	28.85	19.91	22.78
	DemLearn				30.75	28.31	22.25	27.10
	HierMo				19.55	19.59	20.30	19.81
	HierQSGD				19.59	19.59	19.59	19.59
	FedAgg	M_{end}^1	M_{edge}	M_{cloud}	72.05	78.54	79.67	76.75
	FedEEC				80.33	79.98	81.85	80.72
CIFAR-10	HierFAVG	M_{end}^1			19.23	14.64	19.14	17.67
	DemLearn				30.71	27.66	20.64	26.34
	HierMo				16.94	10.78	16.12	14.61
	HierQSGD				13.85	13.22	10.00	12.36
	FedAgg	M_{end}^1	M_{edge}	M_{cloud}	31.67	33.66	34.61	33.31
	FedEEC				33.46	33.96	35.42	34.28
CINIC-10	HierFAVG	M_{end}^1			12.32	11.40	10.91	11.54
	HierMo				12.08	11.01	16.24	13.11
	HierQSGD				13.65	16.07	12.09	13.94
	FedAgg	M_{end}^1	M_{edge}	M_{cloud}	15.24	16.58	17.99	16.60
	FedEEC				16.96	17.16	18.44	17.52

3) *Effectiveness of Self-Knowledge Rectification*: The effectiveness of SKR mechanism is evident when comparing FedEEC (with SKR) with FedAgg (without SKR) in Table III. As demonstrated by the superior performance of FedEEC, SKR plays a crucial role in ensuring reliable knowledge transfer through the hierarchy. By mitigating the impact of misleading attributes within the knowledge, SKR improves the overall quality of the knowledge transferred, resulting in more accurate and stable model updates across tiers.

4) *Convergence Rate*: Figure 5 illustrates the convergence rate of FedEEC compared to the baseline algorithms. Over the course of 50 communication rounds, FedEEC consistently converges faster than all other methods. This can be attributed to its knowledge agglomeration process, which allows the model scale growth across tiers, and the SKR mechanism, which ensures robust knowledge transfer among neighboring nodes. These features contribute to FedEEC’s ability to collaboratively train larger models than those typically supported by end devices, while also reducing the propagation of suboptimal updates throughout the hierarchy.

VI. ABLATION STUDY

In the following subsections, we use the same default hyperparameter configurations as described in the main experiments unless explicitly stated otherwise.

A. Impact of Distillation Weight

We evaluate the effectiveness of FedEEC across various values of the distillation weight β . As shown in Figure 6 (a) and Table IV, FedEEC maintains a clear advantage over

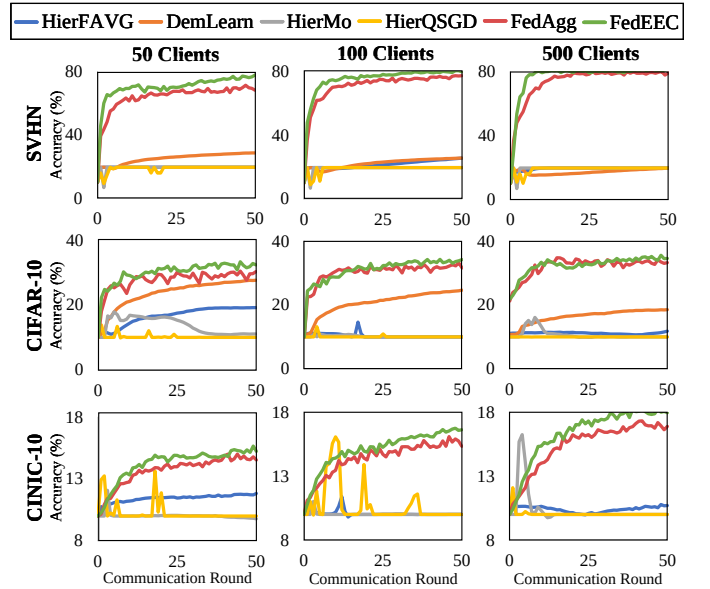


Fig. 5. Learning curves illustrating cloud model accuracy (%) across communication rounds.

FedAgg across the entire range of β values (from 0.3 to 50) with only minor performance fluctuations. This stability highlights FedEEC’s robustness to variations in β , reducing the necessity for extensive hyperparameter tuning to achieve strong performance.

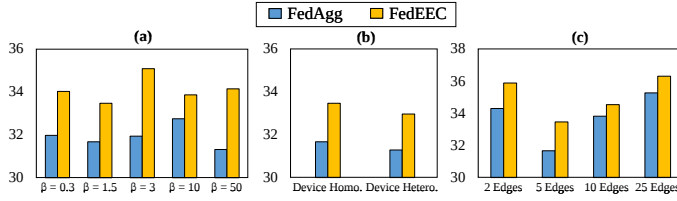


Fig. 6. Cloud model accuracy (%) of FedAgg and FedEEC over different β , on-device model setting and edge numbers.

TABLE IV
IMPACT OF DISTILLATION WEIGHT β ON CLOUD MODEL ACCURACY. RESULTS ARE OBTAINED ON CIFAR-10 DATASET WITH $K = 100$.

Method	$\beta = 0.3$	$\beta = 1.5$	$\beta = 3$	$\beta = 10$	$\beta = 50$
FedAgg	31.97	31.67	31.93	32.74	31.31
FedEEC	34.01	33.46	35.07	33.85	34.13

B. Tolerance to Device Heterogeneity

We evaluate the robustness of FedEEC in environments with two settings of device heterogeneity. As illustrated in Figure 6 (b) and Table V, FedEEC consistently outperforms FedAgg across both homogeneous and heterogeneous device configurations. This highlights FedEEC’s adaptability to environments with varying computational capabilities across devices, making it particularly suitable for real-world applications where seamless collaboration among differentiated on-device models is essential.

C. Impact of Edge Number

We further assess the scalability of FedEEC by varying the number of edge nodes, as shown in Figure 6 (c) and Table VI. Across all edge configurations, FedEEC consistently achieves higher cloud model accuracy than FedAgg. These results demonstrate that FedEEC not only makes sufficient utilization of edge-side resources but also maintains robust performance across diverse network topologies.

VII. DISCUSSIONS

A. Analysis on Communication Overhead

Communication overhead is a critical factor in FL, especially in hierarchical settings like EECC. In Table VII, we compare the network bandwidth consumption of HierFAVG and FedEEC. As shown, FedEEC demonstrates significantly better communication efficiency, reducing network bandwidth usage by 91.57% on average in end-edge communication and 15.66% in edge-cloud communication. This improvement is primarily due to FedEEC’s strategy of exchanging lightweight logits and embeddings instead of full model parameters, which is the standard approach in prevailing HFL methods including HierFAVG. By minimizing the size of transmitted information during training, FedEEC not only achieves robust model performance but also effectively reduces communication overhead across the network hierarchy.

TABLE V
IMPACT OF DIFFERENT ON-DEVICE MODEL HETEROGENEITY SETUP ON CLOUD MODEL ACCURACY. THE M_{end}^1 AND M_{end}^2 EACH CONSTITUTE HALF OF THE END-SIDE MODELS IN THE DEVICE HETEROGENEITY SETUP.

Method	Device Homo.	Device Hetero.
FedAgg	31.67	31.29
FedEEC	33.46	32.96

TABLE VI
IMPACT OF DIFFERENT EDGE NODE NUMBERS ON CLOUD MODEL ACCURACY.

Method	2 Edges	5 Edges	10 Edges	25 Edges
FedAgg	34.29	31.67	33.81	35.26
FedEEC	35.87	33.46	34.53	36.29

B. Limitations

One limitation of FedEEC is its reliance on the pre-trained autoencoder for generating bridge samples. On the one hand, the degree to which the autoencoder aligns with the data distribution on end devices may impact the quality of the generated embeddings as well as synthetic samples, thereby affecting the overall effectiveness of the online distillation process. On the other hand, the need for large-scale pre-training introduces a dependency that may not be practical in all deployment scenarios. Another limitation of FedEEC is that the performance of these upper-tier models is fundamentally constrained by the knowledge distilled from the resource-constrained end devices. When the end devices have extremely limited computational resources, the quality of the knowledge they produce may not be sufficient to fully exploit the capacity of the larger models deployed at upper tiers. Addressing these limitations would significantly enhance FedEEC’s applicability across wider range of deployment contexts.

VIII. RELATED WORKS

A. Hierarchical Federated Learning

HFL leverages multi-tier tree topologies to manage the training process of conventional FL, coordinating clients through a central server (typically located in the cloud) and multiple intermediate computing nodes (typically located at the edge) [40]. Liu et al. [11] introduced the pioneering HFL framework that facilitated faster model training and reduced edge-cloud communication costs by enabling partial model aggregation at edge servers. Subsequent enhancements include the application of quantization techniques to reduce bandwidth usage [41] and the incorporation of momentum acceleration to speed up convergence [14]. Research conducted by Nguyen et al. [25] and Wang et al. [13] explored self-organizing structures and dynamic training topologies in HFL to improve generalization performance on heterogeneous data and reduce completion time over heterogeneous devices, respectively. Security enhancements in HFL have been addressed by Guo et al. [42], who implemented a two-tier differential privacy mechanism to balance efficiency and privacy.

TABLE VII

COMPARISON OF COMMUNICATION OVERHEAD BETWEEN HIERFAVG AND FEDEEC. r AND E REPRESENT THE COMMUNICATION ROUND AND THE NUMBER OF EDGE SERVERS, RESPECTIVELY. BOTH METHODS ARE EVALUATED UNDER THE GOAL OF TRAINING A MODEL WITH IDENTICAL STRUCTURE M_{cloud} ON THE CLOUD.

End-Edge Communication	Method	Communication Complexity	SVHN	CIFAR-10	CINIC-10	Average
	HierFAVG	$O(r \cdot \sum_{i=1}^K W^i)$	208.20G	208.20G	208.20G	208.20G
	FedEEC	$O(\sum_{k=1}^K \mathcal{D}^k \cdot (\varepsilon + 1 + r \cdot (z^\varepsilon + 1)))$	18.10G	12.35G	22.24G	17.56G
Edge-Cloud Communication	Method	Communication Complexity	SVHN	CIFAR-10	CINIC-10	Average
	HierFAVG	$O(r \cdot \sum_{i=1}^E W^i)$	20.82G	20.82G	20.82G	20.82G
	FedEEC	$O(\sum_{k=1}^K \mathcal{D}^k \cdot (\varepsilon + 1 + r \cdot (z^\varepsilon + 1)))$	18.10G	12.35G	22.24G	17.56G

B. Training Larger Model in Federated Learning

Prevailing FL methods [5]–[7] typically employ homogeneous models across diverse computing nodes, which restricts the training of large-scale models due to the bottleneck effect imposed by resource-constrained devices. To overcome this limitation, techniques such as knowledge distillation and partial training have been employed to transfer insights from smaller models trained on end devices to larger models on remote servers. Partial training-based approaches [22], [31] divide the complete global model into multiple sub-models, training these sub-models on multiple clients in parallel. This allows the collaboratively trained server-side model to exceed the size of the largest model on any individual client. Knowledge distillation-based approaches [21], [23], [26] use the model outputs from clients as a regularization term for server-side model training, facilitating the transfer of integrated representations learned by clients on decentralized private data.

C. Knowledge Distillation Empowered Federated Learning for Edge Computing

Knowledge distillation has become increasingly significant in addressing challenges encountered when deploying FL in edge computing scenarios [43], such as user personalization, resource heterogeneity, and communication efficiency. Personalized or multi-task FL approaches proposed by Wu et al. [44] and Jin et al. [45] leverage device-specific distillation to enhance model personalization performance on edge devices. Global knowledge distillation methods proposed by Yao et al. [46] mitigate client drift by using historical global models as teachers to guide local model training. Liu et al. [47] combined FedAvg with block-wise regularization and knowledge distillation to enhance both model performance and communication efficiency. Other notable advancements include knowledge cache-driven FL [48], [49], which significantly improves training communication efficiency while maintaining personalization accuracy.

IX. CONCLUSION

In this paper, we propose End-Edge-Cloud Federated Learning with Self-Rectified Knowledge Agglomeration (FedEEC), a novel framework that overcomes the key limitations of conventional Hierarchical Federated Learning (HFL) within End-Edge-Cloud Collaboration (EECC) environments. FedEEC effectively tackles critical challenges, including scalable model training across multi-tier computing nodes, effective hierarchical knowledge transfer, and support for dynamic node migration. By introducing the Bridge Sample-Based Online Distillation Protocol (BSBODP) and Self-Knowledge Rectification (SKR), FedEEC facilitates model growth in size and generalization ability across tiers, while simultaneously refining the transferred knowledge to mitigate the propagation of misleading predictions and enhances cloud model optimization. Additionally, FedEEC allows for flexible parent switching over non-root computing nodes, making the framework highly adaptive and resilient to a wide range of resource-dynamic environments. Extensive experiments across diverse datasets and configurations demonstrate that FedEEC consistently outperforms existing methods, achieving superior cloud model accuracy while maintaining communication efficiency.

REFERENCES

- [1] statista, “Number of internet of things (iot) connections worldwide from 2022 to 2023, with forecasts from 2024 to 2033.” [Online]. Available: <https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/>
- [2] E. Baccour, N. Mhaisen, A. A. Abdellatif, A. Erbad, A. Mohamed, M. Hamdi, and M. Guizani, “Pervasive ai for iot applications: A survey on resource-efficient distributed artificial intelligence,” *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2366–2418, 2022.
- [3] Q. Yang, Y. Liu, T. Chen, and Y. Tong, “Federated machine learning: Concept and applications,” *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 10, no. 2, pp. 1–19, 2019.
- [4] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, “Federated learning for internet of things: A comprehensive survey,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1622–1658, 2021.
- [5] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

- [6] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 429–450, 2020.
- [7] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *International conference on machine learning*. PMLR, 2020, pp. 5132–5143.
- [8] S. Duan, D. Wang, J. Ren, F. Lyu, Y. Zhang, H. Wu, and X. Shen, "Distributed artificial intelligence empowered by end-edge-cloud computing: A survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 591–624, 2022.
- [9] Y. Wang, C. Yang, S. Lan, L. Zhu, and Y. Zhang, "End-edge-cloud collaborative computing for deep learning: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, 2024.
- [10] G. Bao and P. Guo, "Federated learning in cloud-edge collaborative architecture: key technologies, applications and challenges," *Journal of Cloud Computing*, vol. 11, no. 1, p. 94, 2022.
- [11] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *ICC 2020-2020 IEEE International Conference on Communications (ICC)*. IEEE, 2020, pp. 1–6.
- [12] A. Tak and S. Cherkaoui, "Federated edge learning: Design issues and challenges," *IEEE Network*, vol. 35, no. 2, pp. 252–258, 2020.
- [13] Z. Wang, H. Xu, J. Liu, Y. Xu, H. Huang, and Y. Zhao, "Accelerating federated learning with cluster construction and hierarchical aggregation," *IEEE Transactions on Mobile Computing*, vol. 22, no. 7, pp. 3805–3822, 2023.
- [14] Z. Yang, S. Fu, W. Bao, D. Yuan, and A. Y. Zomaya, "Hierarchical federated learning with momentum acceleration in multi-tier networks," *IEEE Transactions on Parallel and Distributed Systems*, 2023.
- [15] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Hierarchical federated learning with quantization: Convergence analysis and system design," *IEEE Transactions on Wireless Communications*, vol. 22, no. 1, pp. 2–18, 2023.
- [16] S. Wang, R. Ugaonkar, M. Zafer, T. He, K. Chan, and K. K. Leung, "Dynamic service migration in mobile edge computing based on markov decision process," *IEEE/ACM Transactions on Networking*, vol. 27, no. 3, pp. 1272–1288, 2019.
- [17] T. Ouyang, Z. Zhou, and X. Chen, "Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing," *IEEE Journal on Selected Areas in Communications*, vol. 36, no. 10, pp. 2333–2345, 2018.
- [18] Y. Deng, J. Ren, C. Tang, F. Lyu, Y. Liu, and Y. Zhang, "A hierarchical knowledge transfer framework for heterogeneous federated learning," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.
- [19] A. Afonin and S. P. Karimireddy, "Towards model agnostic federated learning using knowledge distillation," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=IQI_mZjvBxj
- [20] S. Cheng, J. Wu, Y. Xiao, and Y. Liu, "Fedgems: Federated learning of larger server models via selective knowledge fusion," *arXiv preprint arXiv:2110.11027*, 2021.
- [21] Y. J. Cho, A. Manoel, G. Joshi, R. Sim, and D. Dimitriadis, "Heterogeneous ensemble knowledge transfer for training large models in federated learning," in *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*, L. D. Raedt, Ed. International Joint Conferences on Artificial Intelligence Organization, 7 2022, pp. 2881–2887, main Track. [Online]. Available: <https://doi.org/10.24963/ijcai.2022/399>
- [22] S. Alam, L. Liu, M. Yan, and M. Zhang, "Fedrolex: Model-heterogeneous federated learning with rolling sub-model extraction," *Advances in neural information processing systems*, vol. 35, pp. 29 677–29 690, 2022.
- [23] C. He, M. Annavaram, and S. Avestimehr, "Group knowledge transfer: Federated learning of large cnns at the edge," *Advances in Neural Information Processing Systems*, vol. 33, pp. 14 068–14 080, 2020.
- [24] Z. Wu, S. Sun, Y. Wang, M. Liu, Q. Pan, J. Zhang, Z. Li, and Q. Liu, "Exploring the distributed knowledge congruence in proxy-data-free federated distillation," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 2, pp. 1–34, 2024.
- [25] M. N. H. Nguyen, S. R. Pandey, T. N. Dang, E.-N. Huh, N. H. Tran, W. Saad, and C. S. Hong, "Self-organizing democratized learning: Toward large-scale distributed learning systems," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 12, pp. 10 698–10 710, 2023.
- [26] Z. Wu, S. Sun, Y. Wang, M. Liu, B. Gao, Q. Pan, T. He, and X. Jiang, "Agglomerative federated learning: Empowering larger model training via end-edge-cloud collaboration," in *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 2024, pp. 131–140.
- [27] Z. Wang, H. Xu, J. Liu, Y. Xu, H. Huang, and Y. Zhao, "Accelerating federated learning with cluster construction and hierarchical aggregation," *IEEE Transactions on Mobile Computing*, vol. 22, no. 7, pp. 3805–3822, 2023.
- [28] R. Anil, G. Pereyra, A. Passos, R. Ormandi, G. E. Dahl, and G. E. Hinton, "Large scale distributed neural network training through online distillation," *arXiv preprint arXiv:1804.03235*, 2018.
- [29] L. Wang and K.-J. Yoon, "Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
- [30] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.
- [31] Q. Nguyen, H. H. Pham, K.-S. Wong, P. Le Nguyen, T. T. Nguyen, and M. N. Do, "Feddict: Federated learning of large convolutional neural networks on resource constrained devices using divide and collaborative training," *IEEE Transactions on Network and Service Management*, 2023.
- [32] C. He, S. Li, J. So, X. Zeng, M. Zhang, H. Wang, X. Wang, P. Vepakomma, A. Singh, H. Qiu *et al.*, "Fedml: A research library and benchmark for federated machine learning," *arXiv preprint arXiv:2007.13518*, 2020.
- [33] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, A. Y. Ng *et al.*, "Reading digits in natural images with unsupervised feature learning," in *NIPS workshop on deep learning and unsupervised feature learning*, vol. 2011, no. 2. Granada, 2011, p. 4.
- [34] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [35] L. N. Darlow, E. J. Crowley, A. Antoniou, and A. J. Storkey, "Cinic-10 is not imagenet or cifar-10," *arXiv preprint arXiv:1810.03505*, 2018.
- [36] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [37] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on non-iid data silos: An experimental study," in *2022 IEEE 38th international conference on data engineering (ICDE)*. IEEE, 2022, pp. 965–978.
- [38] M. N. Nguyen, S. R. Pandey, T. N. Dang, E.-N. Huh, N. H. Tran, W. Saad, and C. S. Hong. [Online]. Available: <https://github.com/nhatminh/Dem-AI>
- [39] L. Liu, J. Zhang, S. Song, and K. B. Letaief. [Online]. Available: https://github.com/LuminLiu/Hier_QSGD
- [40] Q. Duan, J. Huang, S. Hu, R. Deng, Z. Lu, and S. Yu, "Combining federated learning and edge computing toward ubiquitous intelligence in 6g network: Challenges, recent advances, and future directions," *IEEE Communications Surveys & Tutorials*, 2023.
- [41] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Hierarchical federated learning with quantization: Convergence analysis and system design," *IEEE Transactions on Wireless Communications*, vol. 22, no. 1, pp. 2–18, 2023.
- [42] Y. Guo, F. Liu, T. Zhou, Z. Cai, and N. Xiao, "Privacy vs. efficiency: Achieving both through adaptive hierarchical federated learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1331–1342, 2023.
- [43] Z. Wu, S. Sun, Y. Wang, M. Liu, X. Jiang, and R. Li, "Survey of knowledge distillation in federated edge learning," *arXiv preprint arXiv:2301.05849*, 2023.
- [44] Z. Wu, S. Sun, Y. Wang, M. Liu, Q. Pan, X. Jiang, and B. Gao, "Fedict: Federated multi-task distillation for multi-access edge computing," *IEEE Transactions on Parallel and Distributed Systems*, 2023.
- [45] H. Jin, D. Bai, D. Yao, Y. Dai, L. Gu, C. Yu, and L. Sun, "Personalized edge intelligence via federated self-knowledge distillation," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 2, pp. 567–580, 2023.
- [46] D. Yao, W. Pan, Y. Dai, Y. Wan, X. Ding, C. Yu, H. Jin, Z. Xu, and L. Sun, "Fedgkd: Toward heterogeneous federated learning via global knowledge distillation," *IEEE Transactions on Computers*, vol. 73, no. 1, pp. 3–17, 2024.
- [47] J. Liu, Q. Zeng, H. Xu, Y. Xu, Z. Wang, and H. Huang, "Adaptive block-wise regularization and knowledge distillation for enhancing federated learning," *IEEE/ACM Transactions on Networking*, vol. 32, no. 1, pp. 791–805, 2024.
- [48] Z. Wu, S. Sun, Y. Wang, M. Liu, K. Xu, W. Wang, X. Jiang, B. Gao, and J. Lu, "Fedcache: A knowledge cache-driven federated learning

architecture for personalized edge intelligence,” *IEEE Transactions on Mobile Computing*, 2024.

- [49] Q. Pan, S. Sun, Z. Wu, Y. Wang, M. Liu, and B. Gao, “Fedcache 2.0: Exploiting the potential of distilled data in knowledge cache-driven federated learning,” *arXiv preprint arXiv:2405.13378*, 2024.



Zhiyuan Wu (Member, IEEE) is currently a research assistant with the Institute of Computing Technology, Chinese Academy of Sciences (ICT, CAS). He has contributed several technical papers to top-tier conferences and journals as the first author in the fields of computer architecture, computer networks, and intelligent systems, including IEEE Transactions on Parallel and Distributed Systems (TPDS), IEEE Transactions on Mobile Computing (TMC), IEEE International Conference on Computer Communications (INFOCOM), and ACM Transactions on Intelligent Systems and Technology (TIST). He is a member of IEEE, ACM, the China Computer Federation (CCF), and is granted the President Special Prize of ICT, CAS. His research interests include federated learning, mobile edge computing, and distributed systems.

tions on Intelligent Systems and Technology (TIST). He is a member of IEEE, ACM, the China Computer Federation (CCF), and is granted the President Special Prize of ICT, CAS. His research interests include federated learning, mobile edge computing, and distributed systems.



Sheng Sun is currently an associate professor at the Institute of Computing Technology, Chinese Academy of Sciences. She received her bachelor’s degree from Beihang University, and her Ph.D. from the Institute of Computing Technology, Chinese Academy of Sciences. Dr. Sun has led or executed 5 major funded research projects and published over 20 technical papers in journals and conferences related to computer network and distributed systems, including IEEE Transactions on Parallel and Distributed Systems (TPDS), IEEE Transactions on Mobile Computing (TMC), and IEEE International Conference on Computer Communications (INFOCOM). Her research interests include federated learning, edge intelligence, and privacy computing.

Mobile Computing (TMC), and IEEE International Conference on Computer Communications (INFOCOM). Her research interests include federated learning, edge intelligence, and privacy computing.



Yuwei Wang (Member, IEEE) received his Ph.D. degree in computer science from the University of Chinese Academy of Sciences, Beijing, China. He is currently an associate professor at the Institute of Computing Technology, Chinese Academy of Sciences. He has been responsible for setting over 30 international and national standards, and also holds various positions in both international and national industrial standards development organizations (SDOs) as well as local research institutions, including the associate rapporteur at the ITU-T

SG21 Q5, and the deputy director of China Communications Standards Association (CCSA) TC1 WG1. His current research interests include federated learning, mobile edge computing, and next-generation network architecture.



Min Liu (Senior Member, IEEE) received her Ph.D degree in computer science from the Graduate University of the Chinese Academy of Sciences, China. Before that, she received her B.S. and M.S. degrees in computer science from Xi’an Jiaotong University, China. She is currently a professor at the Institute of Computing Technology, Chinese Academy of Sciences, and also holds a position at the Zhongguancun Laboratory. Her current research interests include mobile computing and edge intelligence.



the Editor of IEEE Internet of Things Journal.

Ke Xu (Fellow, IEEE) received the Ph.D. degree from the Department of Computer Science and Technology, Tsinghua University, Beijing, China. He serves as a Full Professor at Tsinghua University. He has published more than 200 technical papers and holds 11 U.S. patents in the research areas of next-generation internet, blockchain systems, the Internet of Things, and network security. He is a member of ACM. He served as the Steering Committee Chair for IEEE/ACM IWQoS. He has guest-edited several special issues in IEEE and Springer journals. He is



Quyang Pan is currently a research assistant with the Institute of Computing Technology, Chinese Academy of Sciences. He has published several technical papers at top-tier conferences and journals in the fields of computer architecture, computer networks, and intelligent systems, including IEEE Transactions on Parallel and Distributed Systems (TPDS), IEEE Transactions on Mobile Computing (TMC), IEEE International Conference on Computer Communications (INFOCOM), and ACM Transactions on Intelligent Systems and Technology (TIST).

He is also an outstanding competitive programmer who has won several gold medals in international and national contests such as ACM International Collegiate Programming Contest (ICPC), CCF Collegiate Computer Systems and Programming Contest (CCSP), etc. His research interests include federated learning and edge computing.



Bo Gao (Member, IEEE) received his M.S. degree in electrical engineering from the School of Electronic Information and Electrical Engineering at Shanghai Jiaotong University, Shanghai, China in 2009, and his Ph.D. degree in computer engineering from the Bradley Department of Electrical and Computer Engineering at Virginia Tech, Blacksburg, USA in 2014. He was an Assistant Professor with the Institute of Computing Technology at Chinese Academy of Sciences, Beijing, China from 2014 to 2017. He was a Visiting Researcher with the School of Computing and Communications at Lancaster University, Lancaster, UK from 2018 to 2019. He is currently an Associate Professor with the School of Computer and Information Technology at Beijing Jiaotong University, Beijing, China. He has directed a number of research projects sponsored by the National Natural Science Foundation of China (NSFC) or other funding agencies. He is a member of IEEE, ACM, and China Computer Federation (CCF). His research interests include wireless networking, mobile/edge computing, multiagent systems, and machine learning.

computing and Communications at Lancaster University, Lancaster, UK from 2018 to 2019. He is currently an Associate Professor with the School of Computer and Information Technology at Beijing Jiaotong University, Beijing, China. He has directed a number of research projects sponsored by the National Natural Science Foundation of China (NSFC) or other funding agencies. He is a member of IEEE, ACM, and China Computer Federation (CCF). His research interests include wireless networking, mobile/edge computing, multiagent systems, and machine learning.



Tian Wen is currently a research assistant with the Institute of Computing Technology, Chinese Academy of Sciences. His research interests include federated learning, edge computing, and information security.